

Unload



Von der Datenbank zum JSON-Endpunkt

nick.schneeberger@fhgr.ch

Inhalt

- Der Datenvertrag: was der Endpunkt liefern muss
- In der Datenbank lesen: `SELECT`, `JOIN` und PDO
- Als JSON antworten: Header und `json_encode`
- Leere Resultate und Fehler beantworten
- Mit `$_GET` filtern

Wo wir stehen



Die Daten liegen bereits richtig in der Datenbank.

Jetzt liefern wir genau die vereinbarte Form an das Frontend.

Was bedeutet «Unload»?

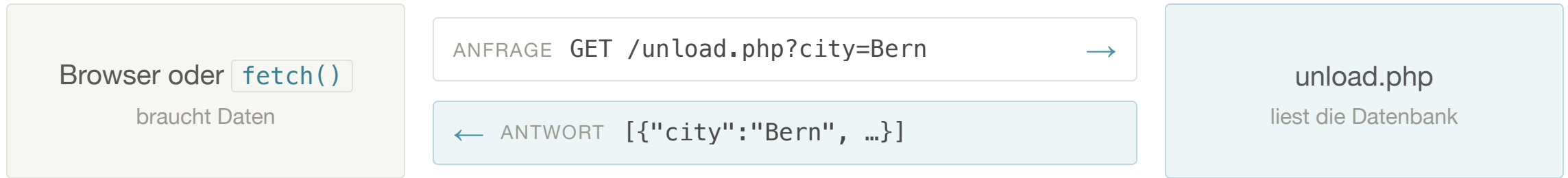
UNSERE KURSBEZEICHNUNG

Daten aus der Datenbank lesen und als JSON ausgeben.

- ETL endet normalerweise beim Laden
- Unser Projekt braucht danach einen Weg zurück ins Frontend
- `unload.php` ist dieser Weg
- Die Datei liefert Daten und kein HTML

«Unload» ist hier ein praktischer Kursbegriff und kein zusätzlicher Buchstabe in ETL.

Der Browser fragt, PHP antwortet mit JSON



Der Browser bekommt am Ende nur JSON – hergestellt hat es unser PHP.

Eine fremde API wie [aareguru](https://aareguru.existenz.ch/v2018/current?city=bern) macht genau dasselbe: URL rein, JSON raus.

Neu ist nur, dass jetzt ihr die Seite mit dem PHP seid.

Beispiel zum Anschauen: aareguru.existenz.ch/v2018/current?city=bern

Der Datenvertrag kommt zuerst

```
[
  {
    "city": "Bern",
    "year": 2023,
    "measurement_days": 92,
    "hot_days": 12,
    "max_temperature_c": 36.3
  }
]
```

Feld	Typ
city	String
year	Number
measurement_days	Number
hot_days	Number
max_temperature_c	Number oder null

Der Endpunkt muss genau diese Form liefern, damit das Frontend ihn ohne Anpassung konsumieren kann.

Vier Bausteine ergeben den Endpunkt

1

Verbinden

PHP mit `config.php` und PDO

2

Lesen

SQL mit `SELECT` und `JOIN`

3

Antworten

PHP mit Header und
`json_encode`

4

Filtern

PHP mit `$_GET`

Jeder Baustein löst genau eine Aufgabe und ist ein Kapitel dieses Foliensatzes.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Bausteine 1 und 2

Aus der Datenbank lesen

Erst die SQL-Abfrage, dann PHP mit PDO

Zwei Tabellen werden eine flache Liste

Datenbank

ZWEI TABELLEN IN DER DATENBANK

cities		
id	name	
1	Bern	

heat_summers		
city_id	year	hot_days
1	2023	12



EIN DATENSATZ IN DER ANTWORT

```
{  
  "city": "Bern",  
  "year": 2023,  
  "hot_days": 12  
}
```

Das Frontend braucht den Namen «Bern» und nicht den Fremdschlüssel `city_id = 1`.

JOIN verbindet passende Zeilen

SQL



JOIN sucht zu jeder Sommerzeile die Stadt mit derselben Nummer.

Nur die vereinbarten Felder lesen

```
SELECT
  c.name AS city,
  hs.year,
  hs.measurement_days,
  hs.hot_days,
  hs.max_temperature_c
FROM heat_summers AS hs
JOIN cities AS c ON c.id = hs.city_id
ORDER BY hs.year, c.name
```

- `AS city` gibt dem Feld direkt den vereinbarten JSON-Namen
- `JOIN` ergänzt den Stadtnamen aus der zweiten Tabelle
- `ORDER BY` macht die Reihenfolge verlässlich
- `SELECT *` bleibt weg, weil der Endpunkt nur vereinbarte Felder ausliefert

PDO macht aus Zeilen ein PHP-Array

PHP

```
require __DIR__ . '/../.../config.php';

$pdo = new PDO($dsn, $username, $password, $options);

$stmt = $pdo->prepare($sql);
$stmt->execute();

$rows = $stmt->fetchAll();
```

`$rows` ist eine Liste assoziativer Arrays.

Die Schlüssel kommen aus dem `SELECT`.

Die Verbindung ist dieselbe wie beim Laden.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Baustein 3

Als JSON antworten

Der Browser bekommt Daten und sonst nichts

Die Antwort sagt, was sie enthält

PHP

```
header('Content-Type: application/json; charset=utf-8');
```

Content-Type

Format der Antwort



application/json

gültiges JSON



charset=utf-8

Zürich bleibt Zürich

Der Header muss vor jeder Ausgabe stehen.

json_encode macht aus dem Array JSON

PHP

PHP-ARRAY

```
[  
    ['city' => 'Bern', 'year' => 2023]  
]
```

→

JSON-TEXT

```
[  
    {"city":"Bern","year":2023}  
]
```

```
echo json_encode($data, JSON_THROW_ON_ERROR | JSON_UNESCAPED_UNICODE);
```

- `JSON_THROW_ON_ERROR` macht Kodierungsfehler sichtbar
- `JSON_UNESCAPED_UNICODE` hält Umlaute lesbar

Kein gültiges JSON

```
var_dump($rows);  
echo '<h1>Fertig</h1>';  
echo json_encode($data);
```

```
array(258) { ... }  
<h1>Fertig</h1>  
[{"city":"Bern",...}]
```

Debug-Text und HTML zerstören die Antwort.

Eine reine Antwort

```
header('Content-Type: application/json; charset=utf-8');  
echo json_encode($data, JSON_THROW_ON_ERROR);
```

```
[{"city":"Bern",...}]
```


Extract → Transform → Load → Datenbank → Unload → Chart.js

Baustein 3 im Ausnahmefall

Leere Resultate und Fehler

Auch der Ausnahmefall antwortet als JSON

Kein Treffer ist kein Fehler

PHP

200 OK

KEINE PASSENDEN ZEILEN

[]

- `fetchAll()` liefert bei null Treffern eine leere Liste
- `json_encode` macht daraus `[]`
- Der Code braucht dafür keine Sonderbehandlung
- Das Frontend kann dieselbe Listenlogik weiterverwenden

Für eine Listenabfrage ist `[]` die stabilste Antwort.

Auch ein Fehler antwortet als JSON

```
try {  
    // verbinden, lesen und antworten  
} catch (Throwable $error) {  
    http_response_code(500);  
    error_log($error->getMessage());  
  
    echo json_encode([  
        'error' => 'Daten konnten nicht geladen werden.',  
    ]);  
}
```

Interne Datenbankmeldungen gehören ins Server-Log und nicht in die öffentliche Antwort.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Baustein 4

Den Endpunkt filtern

Die URL stellt eine kleine Frage an die Daten

\$_GET liest Werte aus der URL

URL + PHP

GET /unload.php? city=Bern

Name

city



Wert

Bern



PHP

\$_GET['city']

```
$city = trim($_GET['city'] ?? '');
```

`?? ''` setzt einen leeren String, wenn der Parameter fehlt.

Der Filter ist optional

PHP + SQL

```
$sql = 'SELECT c.name AS city, hs.year, hs.measurement_days,  
           hs.hot_days, hs.max_temperature_c  
FROM heat_summers AS hs  
JOIN cities AS c ON c.id = hs.city_id';  
  
$params = [];  
  
if ($city !== '') {  
    $sql .= ' WHERE c.name = :city';  
    $params['city'] = $city;  
}  
  
$sql .= ' ORDER BY hs.year, c.name';
```

Ohne `city` kommen alle Städte und mit `city` nur passende Zeilen.

Der Wert bleibt vom SQL getrennt

PHP

```
$statement = $pdo->prepare($sql);  
$statement->execute($params);  
  
$rows = $statement->fetchAll();
```

SQL-BAUPLAN

```
WHERE c.name = :city
```

WERTE

```
['city' => 'Bern']
```

Ohne Filter ist `$params` einfach eine leere Liste.

Vier Aufrufe zeigen, ob der Vertrag hält

Browser

Aufruf	Erwartung
<code>/unload.php</code>	258 sortierte Datensätze
<code>/unload.php?city=Bern</code>	86 Datensätze aus Bern
<code>/unload.php?city=Zürich</code>	86 Datensätze mit korrektem Umlaut
<code>/unload.php?city=Atlantis</code>	leere Liste <code>[]</code>

Prüfe in der Entwicklerkonsole im Netzwerk-Tab zusätzlich den Statuscode und den `Content-Type`.

Übertrag das Muster auf euren Endpunkt

- Nur Felder aus dem Datenvertrag auswählen
- Die JSON-Feldnamen im `SELECT` mit `AS` setzen
- Eine feste und sinnvolle Sortierung setzen
- Einen optionalen Filter als Prepared Statement bauen
- Ohne Treffer eine leere Liste liefern
- Fehler als JSON ohne interne Details beantworten
- Keine HTML- oder Debug-Ausgabe mitsenden
- Mock-JSON und echte Antwort im Team vergleichen

Fertig ist der Endpunkt, wenn das Frontend ohne Sonderlösung zwischen Mock und PHP wechseln kann.

Zum Mitnehmen

`SELECT` liest die Daten in der Datenbank.

`json_encode` liefert sie ans Frontend.

`$_GET` macht den Endpunkt filterbar.

Nächster Schritt: Chart.js lädt den Endpunkt